

System Design: AtliQ Lead Capture and Follow-up Assistant

Version 1.0 · 11 Oct 2026 · Pankaj Mehta, AI Product Manager

Item	Detail
Status	Describes the prototype as built and deployed (GitHub main, Railway)
Related documents	03 AI PRD v1.1 (requirements), 04 LLM Cost Estimate v1.1, 06 HAX Assessment
Diagram format	Archify v3.0.1 (github.com/tt-a1i/archify): typed JSON, validated and rendered to interactive HTML. The HTML files sit next to this document.
Source code	github.com/pankajmehta1827/AtliQ-Lead-Capture

1. Purpose and scope

This document explains how the AtliQ Lead Assistant is built: its components, how a conversation becomes a CRM update, which AI models it calls and why, how data is stored and protected, and how it is deployed and measured. It is written for engineers who will extend the prototype and for reviewers who need to approve it for real client data. The product requirements (what and why) are in the AI PRD; this document covers the how.

2. Context and design goals

Sales owners at AtliQ talk to leads over email, meetings and LinkedIn, but the CRM is updated by hand and falls behind. The assistant reads those conversations, proposes the CRM change with evidence, and leaves the final decision to the owner.

Goal	How the design meets it
Human stays in control	The AI only writes drafts. Nothing reaches the deals table until the owner previews and confirms the change (review gate, HAX G16).
Every field is traceable	Each extracted field carries the quote it came from; fields without a matching quote are dropped at the grounding step.
Private data stays private	Exclusion rules run before anything else and wipe the body of skipped items; PII is masked before any LLM call.
Low running cost	Claude Haiku 5.5 at low effort for capture; the input is capped at 6,000 characters per call; Ask AI stays on Groq.
Works without AI	If no key is set, the same graph runs in rules mode, so the app keeps working.
Simple to run	One container (API, worker, scheduler and built frontend) plus PostgreSQL.

3. Architecture overview

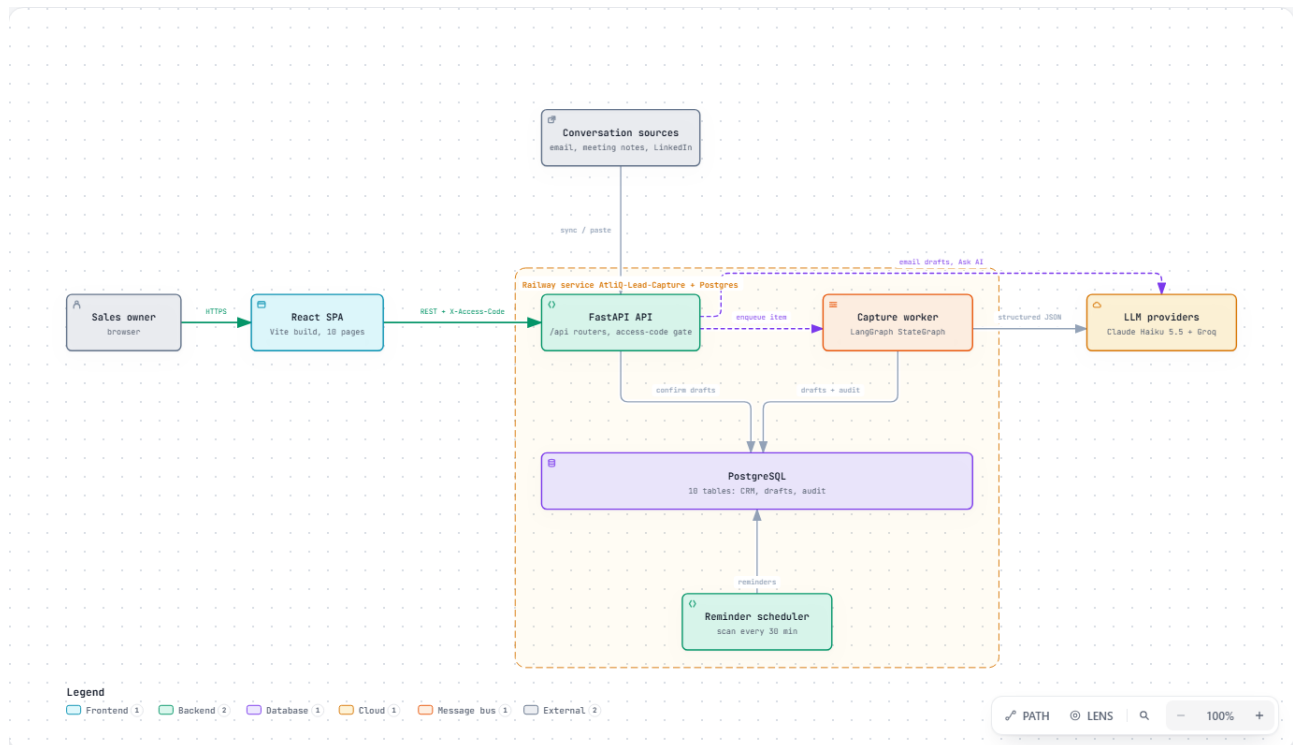


Figure 1. System overview (Archify architecture diagram, interactive version: 07a System Overview.html)

The sales owner uses a React single-page app. It calls a FastAPI backend over REST with an access-code header. Conversations arrive through the API (mailbox sync, file upload or paste) and are stored as queued source items. A capture worker inside the same process picks them up and runs the LangGraph capture pipeline. That pipeline calls the LLM providers for structured JSON and writes drafts and audit entries to PostgreSQL. A reminder scheduler, also in-process, scans deals every 30 minutes and creates follow-up reminders.

3.1 Components

Component	Technology	Responsibility
React SPA	React + TypeScript, Vite	10 pages: Dashboard, Capture, Review Queue, Pipeline, CRM, Deal detail, Reminders, Summary, Ask AI, Settings.
FastAPI API	FastAPI, Pydantic, uvicorn	REST API under /api, access-code check, serves the built SPA and /health.
Capture worker	asyncio task + LangGraph StateGraph	Polls every 5 s and processes up to 5 queued items per cycle, with retries and rate-limit back-off.
Reminder scheduler	asyncio task	Every 30 min: creates rule-based reminders (commitment due, proposal unanswered 7 days, inactive 14 days, no next step).
PostgreSQL	SQLAlchemy 2 (SQLite locally)	CRM tables, capture queue, drafts, reminders, audit log.
LLM providers	Anthropic SDK, LangChain ChatGroq	Claude Haiku 5.5 for capture and email drafts; Groq gpt-oss-120b for Ask AI (and as the capture option).

4. Capture pipeline

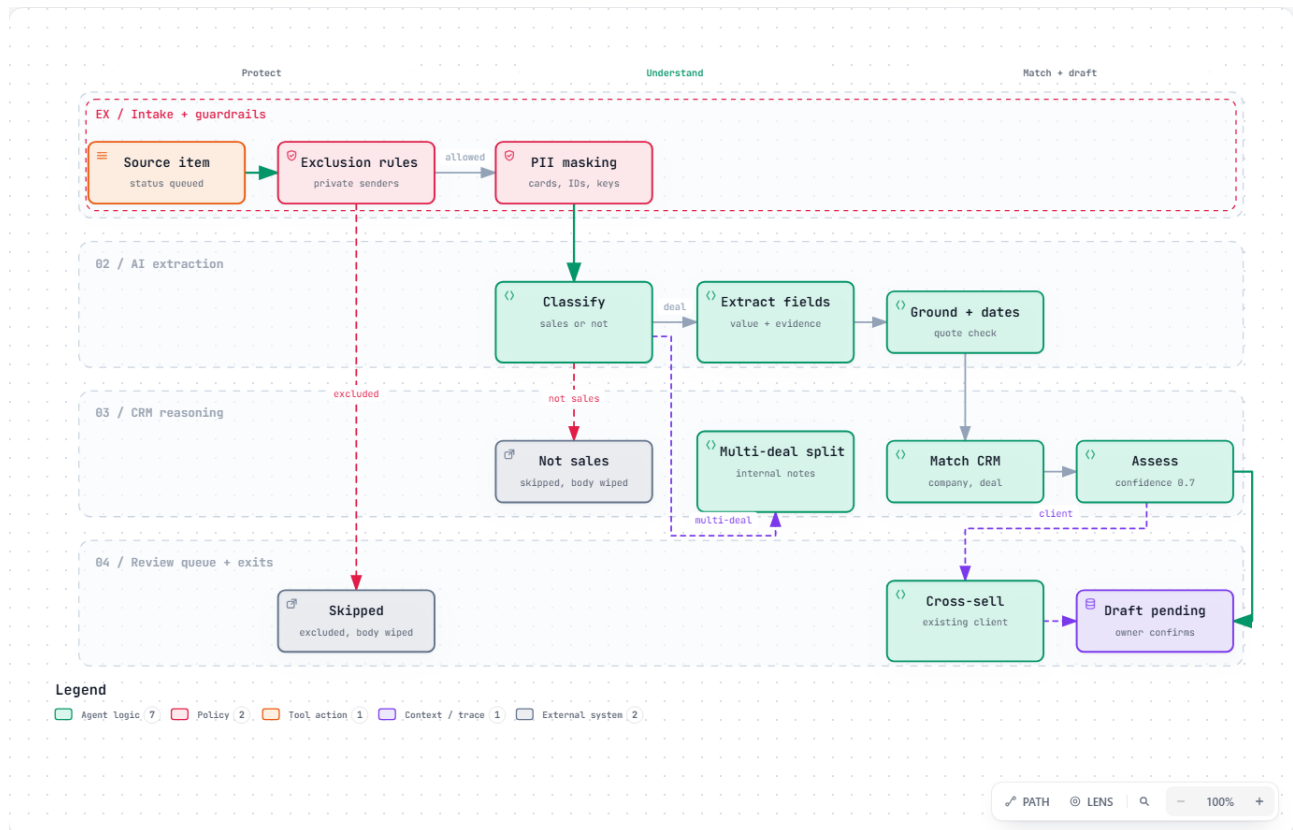


Figure 2. Capture pipeline nodes and branches (Archify workflow diagram, interactive version: [07b Capture Pipeline.html](#))

The pipeline is a LangGraph StateGraph compiled once at start-up (backend/app/graph/pipeline.py). Each node reads and writes a shared state; conditional edges decide when to stop early.

Step	Type	What it does
check_exclusions	Rule	Matches sender email, domain, subject or keyword rules. A match ends the run as skipped and wipes the body.
mask	Rule	Replaces passwords, API keys, IBAN, bank account, IFSC, card, Aadhaar and PAN numbers with labels before any LLM call.
classify	LLM	New lead, existing deal, existing client, internal multi-deal note, or not sales. Not sales ends the run.
extract	LLM	Company, contact, service, requirement, value, owner, next step, each with value, evidence quote and confidence.
multi_extract	LLM	For internal notes that cover several deals: one draft per deal.
ground	Rule	Drops any field whose evidence quote is not found in the conversation.
dates	LLM	Turns 'next Tuesday' style commitments into dates against the message date.
match	Rule	Finds the company and open deal (domain, normalised name); flags duplicates.
assess	Rule	Scores confidence; drafts below 0.7 or with missing fields are marked needs review.
crosssell	LLM	For existing clients only: suggests other AtliQ services with a rationale.

4.1 AI provider design

Each AI job is routed separately through settings, so a model can be swapped without code changes.

AI job	Provider (live)	Setting	Notes
Capture: classify, extract, dates, multi-	Claude Haiku 5.5, effort low	LLM_PROVIDER, CAPTURE_CLAUDE_MODEL	messages.parse with a Pydantic schema, so output is

deal, cross-sell		/ _EFFORT	typed. A refusal becomes a clear item error.
Email drafts	Claude Haiku 5.5, effort low	EMAIL_PROVIDER, EMAIL_CLAUDE_MODEL / _EFFORT	On demand from the deal page. A refusal returns 'please draft it manually'.
Ask AI (questions about the pipeline)	Groq gpt-oss-120b	GROQ_API_KEY, GROQ_MODEL	Always Groq; answers only from CRM data passed in.
No key set	Rules mode	-	Heuristics replace LLM steps; the app still works.

The Claude client pins base_url to api.anthropic.com and uses its own setting names (CAPTURE_CLAUDE_*), so shell variables from developer tools cannot redirect or change it. Temperature is 0 for Groq calls.

5. Data model

Ten tables, created by SQLAlchemy on start-up. JSON columns hold the per-field evidence so the review screen can show where each value came from.

Table	Purpose	Key columns
companies	Client and prospect companies	name, normalized_name, domain
contacts	People at those companies	company_id, name, email
deals	One lead or opportunity	lead_code, status, est_value_usd, owner, next_followup_date, next_step
source_items	Captured conversations (the queue)	channel, external_ref (unique), body, status, attempts, last_error
drafts	Proposed CRM writes awaiting review	kind, fields (value, evidence, confidence), confidence, needs_review, status, owner, llm_mode
deal_activities	Timeline linking a deal to the conversation	deal_id, source_item_id, draft_id, changes
reminders	Follow-ups for the owner	deal_id, kind, due_date, evidence, status
crosssell_suggestions	Service ideas for existing clients	deal_id, service, rationale, status
exclusion_rules	Senders or topics never to process	rule_type, value
audit_log	Every action by a user or the assistant	ts, actor, action, entity, outcome, details

5.1 State lifecycle

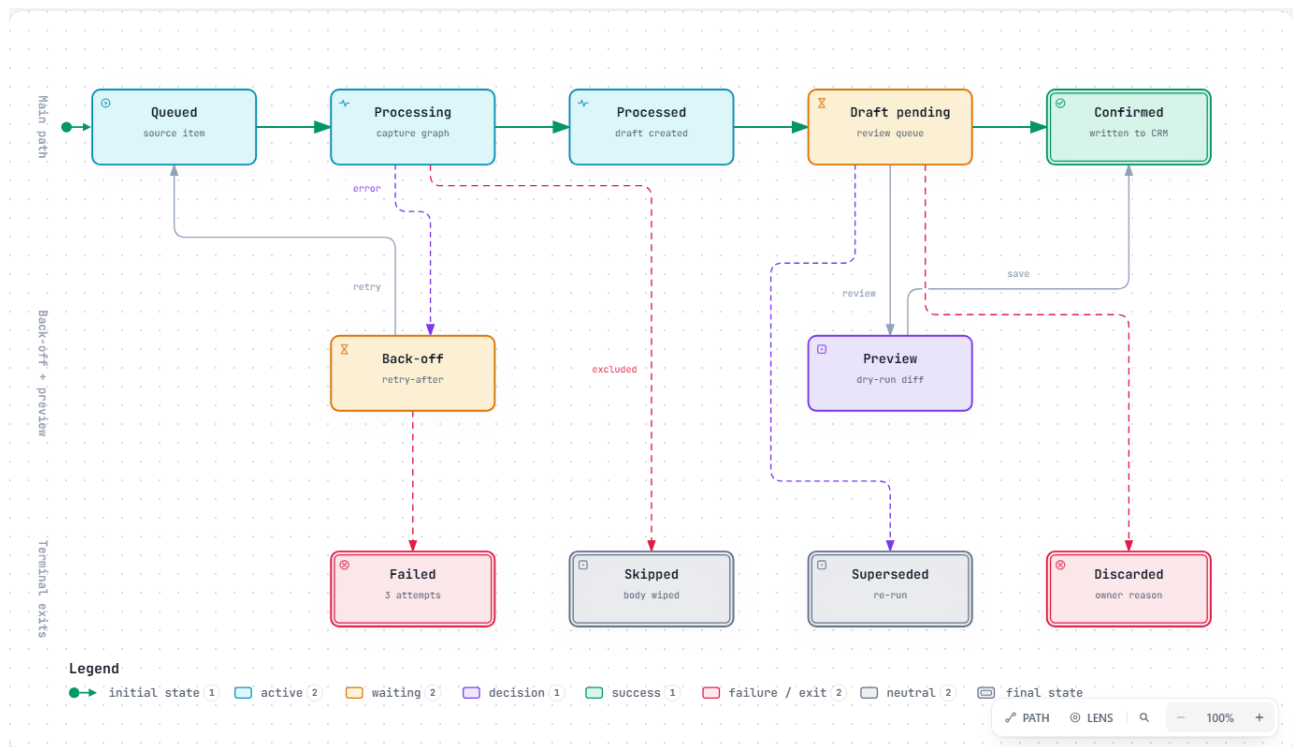


Figure 3. Source item and draft states (Archify lifecycle diagram, interactive version: [07c Item and Draft Lifecycle.html](#))

- Source item: queued → processing → processed, skipped (excluded or not sales, body wiped) or failed (after 3 attempts). Errors and rate limits return the item to the queue after a back-off that honours the provider's retry-after header.
- Draft: pending → confirmed (writes the deal, a timeline activity, reminders and cross-sell ideas) or discarded (reason kept for evals). Re-running an item supersedes its old pending draft.
- Preview is a dry run of confirm: it shows the exact change, then rolls back.

6. API surface

Router	Prefix	Main endpoints
capture	/api/capture	POST " (paste), /upload, /sample (mailbox sync), /process; GET /items; POST /items/{id}/retry; GET/POST /rerun
drafts	/api/drafts	GET list and one; POST /{id}/preview, /{id}/confirm, /{id}/discard
pipeline	/api	GET/PATCH /deals; /reminders (+ scan, resolve); /crosssell; /summary/daily
crm	/api/crm	GET/POST /leads; GET /export; POST /import; GET /sync
ai assist	/api	POST /deals/{id}/email-draft, /ask, /feedback
admin	/api	GET /config, /me, /audit, /metrics; exclusions CRUD; POST /admin/processing

Every request needs the X-User header (the acting seller) and, when APP_ACCESS_CODE is set, a matching X-Access-Code header (constant-time compare).

7. Security and privacy

Risk	Control
Private or personal conversations	Exclusion rules run first; skipped and non-sales items keep no body.

processed	
Sensitive numbers sent to an AI provider	Regex masking of 8 PII and secret types before any LLM call.
AI writes wrong data to the CRM	Drafts only; owner preview and confirm; evidence quote per field; grounding drops unsupported fields.
Unauthorised access	Shared access code over HTTPS (prototype). Replace with SSO before real client data.
Secrets leaking	Keys only in environment variables (Railway variables, local .env not committed).
No accountability	audit_log records every capture, confirm, discard, edit, reminder scan and AI feedback.

8. Reliability and error handling

- Retries: an item that fails is re-queued up to 3 attempts, then marked failed with a friendly error the user can retry from the Capture page.
- Rate limits: the worker pauses for the provider's retry-after time instead of failing items.
- Refusals: a Claude refusal (stop_reason refusal) is reported as a clear error, never as an empty draft.
- Idempotency: external_ref is unique, so syncing the same mailbox twice does not create duplicates.
- Degraded mode: with no AI key the rules pipeline keeps capture running at lower quality.
- Processing can be paused from Settings (POST /api/admin/processing) without stopping the app.

9. Deployment and environments

Environment	How it runs	Database
Local, no Docker	python backend/run_local.py (API + built frontend)	SQLite file
Local, Docker	docker compose up (app + postgres)	PostgreSQL container
Production prototype	Railway service AtliQ-Lead-Capture, auto-deploys from GitHub main via the Dockerfile	Railway PostgreSQL

The Dockerfile builds the frontend and serves it from FastAPI, so there is one service to deploy. Configuration is environment variables only (see .env.example).

Live demo: <https://atliq-lead-capture-production.up.railway.app>

10. Observability and evaluation

- GET /health for uptime checks; start-up log shows which provider handles capture, email drafts and Ask AI.
- GET /api/metrics: North Star (new leads confirmed within 24 hours with source, requirement and next step), minor-edit rate on confirmed drafts, open deals with a next step, inactive deals and review-queue age.
- AI feedback (thumbs up or down) is stored in audit_log as ai_feedback.
- Golden eval set: 65 labelled conversations and 12 Ask AI questions, run through the production path by backend/evals/run_golden.py (rules or LLM mode, resumable).

Eval measure (Groq run, 41 of 65)	Result	Target
Sales detection precision / recall	97.2% / 100%	≥ 95%
CRM matching (wrong merges)	100% (0)	0 wrong merges
Field accuracy	82.8%	90%

Follow-up date accuracy	90.3%	90%
Private data leaks	0	0

Known gap: threads longer than 6,000 characters are cut, so late facts in long threads can be missed.

11. Scaling and next steps

Area	Today	When it is needed
Worker	In-process async task, 5 items per cycle	Move to a separate Railway worker service when volume passes a few thousand items a day.
Queue	source_items table polled by status	Use SKIP LOCKED or a message queue once there is more than one worker.
Mailbox	Sample mailbox sync and upload	Gmail / Outlook connector with per-user OAuth.
Long threads	6,000-character cap	Chunk and summarise threads before extraction.
Authentication	Shared access code	SSO with per-user roles before real client data.
Cost control	Low effort, capped input	Prompt caching for the fixed system prompts; batch API for back-fills.

12. About the diagrams

The three diagrams were authored in the Archify format: each is a typed JSON file (architecture, workflow and lifecycle schemas) that Archify validates, renders to a standalone HTML page with inline SVG, and checks in a real browser. All three passed the validate, deliver, check and browser-check gates. The HTML versions support light and dark themes, path tracing and export to PNG or SVG; the JSON sources are kept in the diagrams folder so they can be edited and re-rendered.

Archify: <https://github.com/tt-a1i/archify>